

```
# 1
import tensorflow as tf
device_name = tf.test.gpu_device_name()
if device_name != '/device:GPU:0':
    raise SystemError('GPU device not found!')
print('Found GPU at: {}'.format(device_name))
```

```
Found GPU at: /device:GPU:0
```

```
# 2
!pip install -q fiftyone
print("Dependencies installed successfully.")
```

```
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
----- 18.0/18.0 MB 94.5 MB/s eta 0:00:00
----- 323.4/323.4 kB 33.8 MB/s eta 0:00:00
----- 43.8/43.8 kB 5.0 MB/s eta 0:00:00
----- 140.5/140.5 kB 11.4 MB/s eta 0:00:00
----- 112.9/112.9 kB 14.0 MB/s eta 0:00:00
----- 44.8/44.8 kB 4.8 MB/s eta 0:00:00
----- 61.6/61.6 kB 7.1 MB/s eta 0:00:00
----- 112.5/112.5 kB 12.5 MB/s eta 0:00:00
----- 74.8/74.8 kB 8.5 MB/s eta 0:00:00
----- 9.9/9.9 MB 107.9 MB/s eta 0:00:00
----- 101.6/101.6 kB 12.0 MB/s eta 0:00:00
----- 1.9/1.9 MB 79.7 MB/s eta 0:00:00
----- 316.4/316.4 kB 34.5 MB/s eta 0:00:00
----- 935.9/935.9 kB 65.9 MB/s eta 0:00:00
----- 15.1/15.1 MB 110.2 MB/s eta 0:00:00
----- 331.1/331.1 kB 33.7 MB/s eta 0:00:00
----- 207.3/207.3 kB 23.4 MB/s eta 0:00:00
----- 227.3/227.3 kB 25.3 MB/s eta 0:00:00
----- 88.6/88.6 kB 10.3 MB/s eta 0:00:00
----- 71.3/71.3 kB 8.4 MB/s eta 0:00:00
----- 495.3/495.3 kB 46.8 MB/s eta 0:00:00
----- 278.2/278.2 kB 32.1 MB/s eta 0:00:00
----- 100.6/100.6 kB 11.4 MB/s eta 0:00:00
----- 51.5/51.5 kB 5.5 MB/s eta 0:00:00
----- 1.4/1.4 MB 86.7 MB/s eta 0:00:00
----- 144.3/144.3 kB 17.3 MB/s eta 0:00:00
```

```
Building wheel for fiftyone-db (pyproject.toml) ... done
ERROR: pip's dependency resolver does not currently take into account all the packages that are instal
google-ai-generativelanguage 0.6.15 requires protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.
grpcio-status 1.71.2 requires protobuf<6.0dev,>=5.26.1, but you have protobuf 6.33.6 which is incompat
Dependencies installed successfully.
```

```
# 3
import fiftyone as fo
import fiftyone.zoo as foz

TARGET_CLASSES = ["Person", "Laptop", "Chair"]

if "open-images-v7-train-1500" in fo.list_datasets():
    fo.delete_dataset("open-images-v7-train-1500")

print("Streaming training split from Open Images V7 (1500 samples)")
train_dataset = foz.load_zoo_dataset(
    "open-images-v7",
    split="train",
    label_types="detections",
    classes=TARGET_CLASSES,
    max_samples=1500,
    dataset_name="open-images-v7-train-1500",
```

```

        drop_existing=True
    )

    print("\nDownloading validation split")
    val_dataset = foz.load_zoo_dataset(
        "open-images-v7",
        split="validation",
        label_types="detections",
        classes=TARGET_CLASSES,
        max_samples=100,
        dataset_name="open-images-v7-validation-100",
        drop_existing=True
    )

    print("\nData layer fully synchronized!")

```

```

/usr/local/lib/python3.12/dist-packages/glob2/fnmatch.py:141: SyntaxWarning: invalid escape sequence
    return '({ms})' + res + '\Z'
Streaming training split from Open Images V7 (1500 samples)
Downloading split 'train' to '/root/fiftyone/open-images-v7/train' if necessary
INFO:fiftyone.zoo.datasets:Downloading split 'train' to '/root/fiftyone/open-images-v7/train' if nece
Downloading 'https://storage.googleapis.com/openimages/2018_04/train/train-images-boxable-with-rotati
INFO:fiftyone.utils.openimages:Downloading 'https://storage.googleapis.com/openimages/2018_04/train/t
100% |██████████| 4.8Gb/4.8Gb [7.3s elapsed, 0s remaining, 669.8Mb/s]
INFO:eta.core.utils: 100% |██████████| 4.8Gb/4.8Gb [7.3s elapsed, 0s remaining, 669.8Mb/s]
Downloading 'https://storage.googleapis.com/openimages/v5/class-descriptions-boxable.csv' to '/root/f
INFO:fiftyone.utils.openimages:Downloading 'https://storage.googleapis.com/openimages/v5/class-descri
Downloading 'https://storage.googleapis.com/openimages/2018_04/bbox_labels_600_hierarchy.json' to '/t
INFO:fiftyone.utils.openimages:Downloading 'https://storage.googleapis.com/openimages/2018_04/bbox_la
Downloading 'https://storage.googleapis.com/openimages/v6/oidv6-train-annotations-bbox.csv' to '/root
INFO:fiftyone.utils.openimages:Downloading 'https://storage.googleapis.com/openimages/v6/oidv6-train-
Downloading 1500 images
INFO:fiftyone.utils.openimages:Downloading 1500 images
100% |██████████| 1500/1500 [3.0m elapsed, 0s remaining, 7.7 files/s]
INFO:eta.core.utils: 100% |██████████| 1500/1500 [3.0m elapsed, 0s remaining, 7.7 files/s]
Dataset info written to '/root/fiftyone/open-images-v7/info.json'
INFO:fiftyone.zoo.datasets:Dataset info written to '/root/fiftyone/open-images-v7/info.json'
Ignoring unsupported parameter 'drop_existing' for importer type <class 'fiftyone.utils.openimages.Op
WARNING:fiftyone.zoo.datasets:Ignoring unsupported parameter 'drop_existing' for importer type <class
Loading 'open-images-v7' split 'train'
INFO:fiftyone.zoo.datasets:Loading 'open-images-v7' split 'train'
100% |██████████| 1500/1500 [17.6s elapsed, 0s remaining, 114.2 samples/s]
INFO:eta.core.utils: 100% |██████████| 1500/1500 [17.6s elapsed, 0s remaining, 114.2 samples/s]
Dataset 'open-images-v7-train-1500' created
INFO:fiftyone.zoo.datasets:Dataset 'open-images-v7-train-1500' created

Downloading validation split
Downloading split 'validation' to '/root/fiftyone/open-images-v7/validation' if necessary
INFO:fiftyone.zoo.datasets:Downloading split 'validation' to '/root/fiftyone/open-images-v7/validatic
Downloading 'https://storage.googleapis.com/openimages/2018_04/validation/validation-images-with-rotat
INFO:fiftyone.utils.openimages:Downloading 'https://storage.googleapis.com/openimages/2018_04/validat
Downloading 'https://storage.googleapis.com/openimages/v5/class-descriptions-boxable.csv' to '/root/f
INFO:fiftyone.utils.openimages:Downloading 'https://storage.googleapis.com/openimages/v5/class-descri
Downloading 'https://storage.googleapis.com/openimages/2018_04/bbox_labels_600_hierarchy.json' to '/t
INFO:fiftyone.utils.openimages:Downloading 'https://storage.googleapis.com/openimages/2018_04/bbox_la
Downloading 'https://storage.googleapis.com/openimages/v5/validation-annotations-bbox.csv' to '/root/
INFO:fiftyone.utils.openimages:Downloading 'https://storage.googleapis.com/openimages/v5/validation-a
Downloading 100 images
INFO:fiftyone.utils.openimages:Downloading 100 images
100% |██████████| 100/100 [12.8s elapsed, 0s remaining, 7.5 files/s]
INFO:eta.core.utils: 100% |██████████| 100/100 [12.8s elapsed, 0s remaining, 7.5 files/s]
Dataset info written to '/root/fiftyone/open-images-v7/info.json'
INFO:fiftyone.zoo.datasets:Dataset info written to '/root/fiftyone/open-images-v7/info.json'
Ignoring unsupported parameter 'drop_existing' for importer type <class 'fiftyone.utils.openimages.Op
WARNING:fiftyone.zoo.datasets:Ignoring unsupported parameter 'drop_existing' for importer type <class
Loading 'open-images-v7' split 'validation'
INFO:fiftyone.zoo.datasets:Loading 'open-images-v7' split 'validation'
100% |██████████| 100/100 [1.8s elapsed, 0s remaining, 56.4 samples/s]
INFO:eta.core.utils: 100% |██████████| 100/100 [1.8s elapsed, 0s remaining, 56.4 samples/s]
Dataset 'open-images-v7-validation-100' created

```

INFO:fiftyone.zoo.datasets:Dataset 'open-images-v7-validation-100' created

Data layer fully synchronized!

```
import numpy as np
import cv2
import fiftyone as fo

MAX_LABELED_OBJECTS = 10

def load_pure_dataset(split_name):
    images_list = []
    boxes_list = []
    labels_list = []

    if split_name == "train":
        fo_dataset = fo.load_dataset("open-images-v7-train-1500")
    else:
        fo_dataset = fo.load_dataset("open-images-v7-validation-100")

    class_map = {"Person": 0, "Laptop": 1, "Chair": 2}
    print(f"Parsing actual Kaggle annotations for {split_name} split...")

    for sample in fo_dataset:
        img_path = sample.filepath
        img = cv2.imread(img_path)
        if img is None: continue
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        img = cv2.resize(img, (320, 320)) / 255.0

        real_boxes = np.zeros((MAX_LABELED_OBJECTS, 4), dtype=np.float32)
        real_labels = np.ones((MAX_LABELED_OBJECTS,), dtype=np.int64) * -1

        if sample.ground_truth is not None:
            count = 0
            for detection in sample.ground_truth.detections:
                if count >= MAX_LABELED_OBJECTS: break
                if detection.label in class_map:
                    xmin, ymin, w, h = detection.bounding_box
                    real_boxes[count] = [ymin, xmin, ymin + h, xmin + w]
                    real_labels[count] = class_map[detection.label]
                    count += 1

        images_list.append(img)
        boxes_list.append(real_boxes)
        labels_list.append(real_labels)

    return np.array(images_list, dtype=np.float32), np.array(boxes_list, dtype=np.float32), np.array(
X_train, Y_boxes_train, Y_labels_train = load_pure_dataset("train")
X_val, Y_boxes_val, Y_labels_val = load_pure_dataset("validation")

Y_labels_train[Y_labels_train == -1] = 3
Y_labels_val[Y_labels_val == -1] = 3

print(f"\nData parsed successfully!")
print(f"Train Images Shape: {X_train.shape} | Train Boxes Shape: {Y_boxes_train.shape}")

Parsing actual Kaggle annotations for train split...
Parsing actual Kaggle annotations for validation split...

Data parsed successfully!
Train Images Shape: (1500, 320, 320, 3) | Train Boxes Shape: (1500, 10, 4)
```

```
import tensorflow as tf
from tensorflow.keras import layers, models
```

```

from tensorflow.keras import layers, models

def build_accurate_detector():
    base_model = tf.keras.applications.MobileNetV2(input_shape=(320, 320, 3), include_top=False, weights='imagenet')

    base_model.trainable = True

    fine_tune_at = len(base_model.layers) - 40
    for layer in base_model.layers[:fine_tune_at]:
        layer.trainable = False

    x = base_model.output
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dense(512, activation='relu')(x)
    x = layers.Dropout(0.2)(x)

    box_output = layers.Dense(10 * 4, activation='sigmoid', name='boxes_head')(x)
    box_output = layers.Reshape((10, 4), name='boxes')(box_output)

    class_output = layers.Dense(10 * 4, activation='linear', name='labels_head')(x)
    class_output = layers.Reshape((10, 4), name='labels')(class_output)

    return models.Model(inputs=base_model.input, outputs={'boxes': box_output, 'labels': class_output})

def masked_box_loss(y_true, y_pred):
    raw_loss = tf.square(y_true - y_pred)
    coordinate_sum = tf.reduce_sum(tf.abs(y_true), axis=-1)
    mask = tf.cast(coordinate_sum > 0.0, tf.float32)
    mask = tf.expand_dims(mask, axis=-1)
    return tf.reduce_mean(raw_loss * mask)

custom_model = build_accurate_detector()
custom_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=5e-6),
    loss={
        'boxes': masked_box_loss,
        'labels': tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
    },
    loss_weights={'boxes': 20.0, 'labels': 1.0}
)
print("Synchronized and compiled successfully!")

```

```

/tmp/ipykernel_3120/3304384038.py:5: UserWarning: `input_shape` is undefined or non-square, or `rows`
base_model = tf.keras.applications.MobileNetV2(input_shape=(320, 320, 3), include_top=False, weights='imagenet')
Synchronized and compiled successfully!

```

```

from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

callbacks_list = [
    EarlyStopping(monitor='val_loss', patience=6, restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3, min_lr=1e-7, verbose=1)
]

print("Launching synchronized fine-tuning pass...")
history = custom_model.fit(
    X_train,
    {'boxes': Y_boxes_train, 'labels': Y_labels_train},
    validation_data=(X_val, {'boxes': Y_boxes_val, 'labels': Y_labels_val}),
    epochs=50,
    batch_size=8,
    callbacks=callbacks_list
)
print("\nModel fully trained and best weights restored!")

```

```

Epoch 6/50
188/188 ----- 6s 31ms/step - boxes_loss: 0.0225 - labels_loss: 0.6600 - loss: 1.1102 -
Epoch 7/50
188/188 ----- 6s 31ms/step - boxes_loss: 0.0219 - labels_loss: 0.6448 - loss: 1.0846 -
Epoch 8/50
188/188 ----- 6s 31ms/step - boxes_loss: 0.0215 - labels_loss: 0.6260 - loss: 1.0564 -
Epoch 9/50
188/188 ----- 5s 29ms/step - boxes_loss: 0.0211 - labels_loss: 0.6106 - loss: 1.0308 -
Epoch 10/50
188/188 ----- 5s 26ms/step - boxes_loss: 0.0209 - labels_loss: 0.6000 - loss: 1.0183 -
Epoch 11/50
188/188 ----- 6s 29ms/step - boxes_loss: 0.0208 - labels_loss: 0.5932 - loss: 1.0081 -
Epoch 12/50
188/188 ----- 10s 28ms/step - boxes_loss: 0.0203 - labels_loss: 0.5656 - loss: 0.9711
Epoch 13/50
188/188 ----- 6s 29ms/step - boxes_loss: 0.0200 - labels_loss: 0.5637 - loss: 0.9634 -
Epoch 14/50
188/188 ----- 5s 25ms/step - boxes_loss: 0.0197 - labels_loss: 0.5534 - loss: 0.9482 -
Epoch 15/50
188/188 ----- 5s 27ms/step - boxes_loss: 0.0196 - labels_loss: 0.5468 - loss: 0.9355 -
Epoch 16/50
188/188 ----- 6s 29ms/step - boxes_loss: 0.0192 - labels_loss: 0.5368 - loss: 0.9213 -
Epoch 17/50
188/188 ----- 5s 25ms/step - boxes_loss: 0.0192 - labels_loss: 0.5225 - loss: 0.9053 -
Epoch 18/50
188/188 ----- 5s 27ms/step - boxes_loss: 0.0188 - labels_loss: 0.5135 - loss: 0.8873 -
Epoch 19/50
188/188 ----- 10s 26ms/step - boxes_loss: 0.0187 - labels_loss: 0.5108 - loss: 0.8821
Epoch 20/50
188/188 ----- 5s 28ms/step - boxes_loss: 0.0183 - labels_loss: 0.5007 - loss: 0.8676 -
Epoch 21/50
188/188 ----- 5s 27ms/step - boxes_loss: 0.0182 - labels_loss: 0.4970 - loss: 0.8620 -
Epoch 22/50
188/188 ----- 5s 25ms/step - boxes_loss: 0.0182 - labels_loss: 0.4989 - loss: 0.8565 -
Epoch 23/50
188/188 ----- 5s 28ms/step - boxes_loss: 0.0179 - labels_loss: 0.4778 - loss: 0.8367 -
Epoch 24/50
188/188 ----- 5s 27ms/step - boxes_loss: 0.0177 - labels_loss: 0.4691 - loss: 0.8215 -
Epoch 25/50
188/188 ----- 5s 25ms/step - boxes_loss: 0.0175 - labels_loss: 0.4604 - loss: 0.8094 -
Epoch 26/50
188/188 ----- 6s 30ms/step - boxes_loss: 0.0173 - labels_loss: 0.4476 - loss: 0.7945 -
Epoch 27/50
188/188 ----- 0s 24ms/step - boxes_loss: 0.0162 - labels_loss: 0.4286 - loss: 0.7534
Epoch 27: ReduceLROnPlateau reducing learning rate to 2.499999936844688e-06.
188/188 ----- 5s 25ms/step - boxes_loss: 0.0170 - labels_loss: 0.4473 - loss: 0.7863 -
Epoch 28/50
188/188 ----- 5s 26ms/step - boxes_loss: 0.0170 - labels_loss: 0.4340 - loss: 0.7733 -
Epoch 29/50
188/188 ----- 6s 30ms/step - boxes_loss: 0.0168 - labels_loss: 0.4374 - loss: 0.7716 -
Epoch 30/50
188/188 ----- 0s 24ms/step - boxes_loss: 0.0164 - labels_loss: 0.4220 - loss: 0.7507
Epoch 30: ReduceLROnPlateau reducing learning rate to 1.249999968422344e-06.
188/188 ----- 5s 25ms/step - boxes_loss: 0.0168 - labels_loss: 0.4327 - loss: 0.7686 -
Epoch 30: early stopping
Restoring model weights from the end of the best epoch: 24.

```

Model fully trained and best weights restored!

```

import numpy as np
import cv2
import matplotlib.pyplot as plt

CUSTOM_LABELS = {0: "Person", 1: "Laptop", 2: "Chair", 3: "Background"}

def verify_image(image_path, confidence_floor=0.30, nms_overlap_limit=0.3):
    img_bgr = cv2.imread(image_path)
    if img_bgr is None:
        print(f"Error: Could not locate file {image_path}")
        return

```

```

img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
h_orig, w_orig, _ = img_rgb.shape

img_resized = cv2.resize(img_rgb, (320, 320)) / 255.0
input_tensor = np.expand_dims(img_resized, axis=0)

predictions = custom_model.predict(input_tensor, verbose=0)
pred_boxes = predictions['boxes'][0]
pred_labels = predictions['labels'][0]

boxes, confidences, class_ids = [], [], []

for i in range(len(pred_boxes)):
    logits = pred_labels[i]
    exp_logits = np.exp(logits - np.max(logits))
    probs = exp_logits / np.sum(exp_logits)

    class_id = int(np.argmax(probs[:3]))
    confidence = float(probs[class_id])

    if confidence > confidence_floor:
        ymin, xmin, ymax, xmax = pred_boxes[i]

        x1 = int(xmin * w_orig)
        y1 = int(ymin * h_orig)
        w = int((xmax - xmin) * w_orig)
        h = int((ymax - ymin) * h_orig)

        if w > 20 and h > 20:
            boxes.append([x1, y1, w, h])
            confidences.append(confidence)
            class_ids.append(class_id)

indices = cv2.dnn.NMSBoxes(boxes, confidences, confidence_floor, nms_overlap_limit)

detected_count = 0
if len(indices) > 0:
    for i in indices.flatten():
        x, y, w, h = boxes[i]
        detected_count += 1

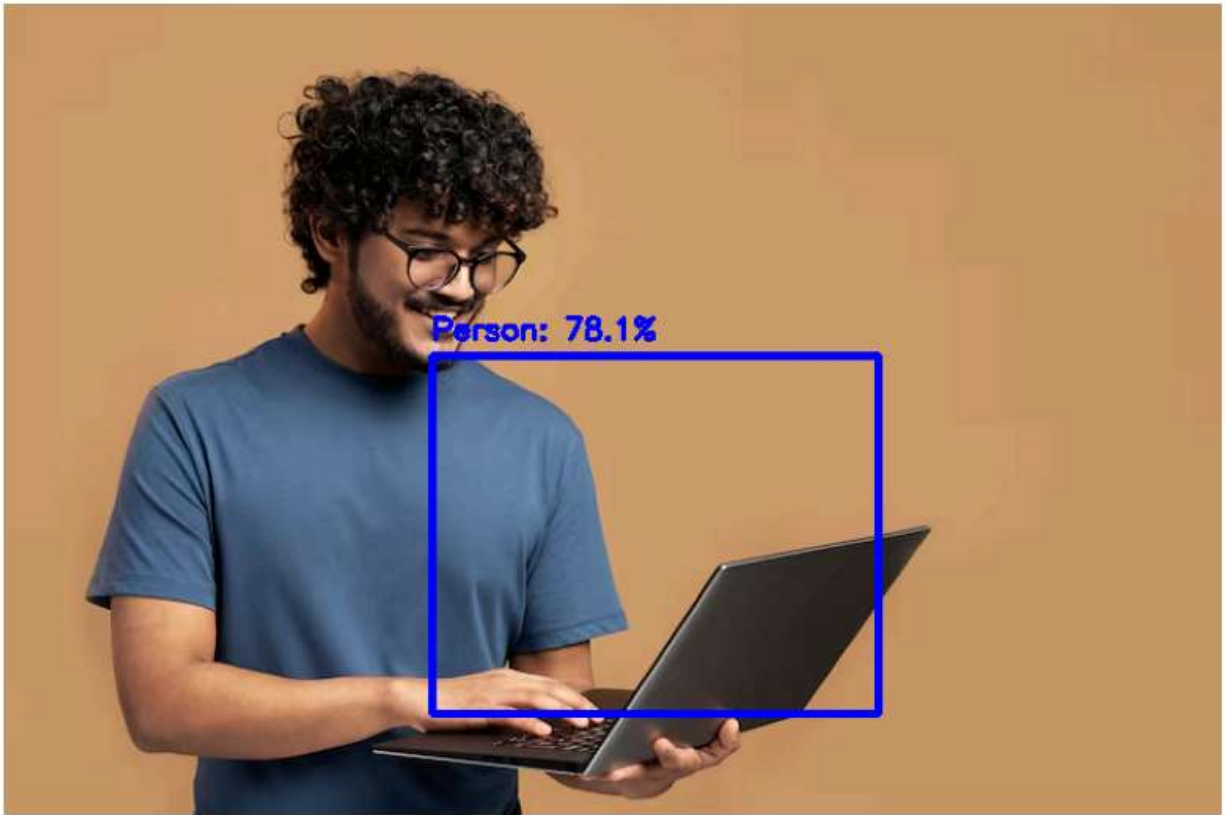
    color_map = {0: (0, 255, 0), 1: (255, 0, 0), 2: (0, 0, 255)}
    box_color = color_map.get(class_ids[i], (255, 255, 255))

    label_text = f"{CUSTOM_LABELS[class_ids[i]]}: {confidences[i]*100:.1f}%"
    cv2.rectangle(img_bgr, (x, y), (x + w, y + h), box_color, 3)
    cv2.putText(img_bgr, label_text, (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, box_color, 2)

output_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(10, 10))
plt.imshow(output_rgb)
plt.axis('off')
plt.show()
print(f"Done! Cleaned up and visualized {detected_count} custom localized objects.")

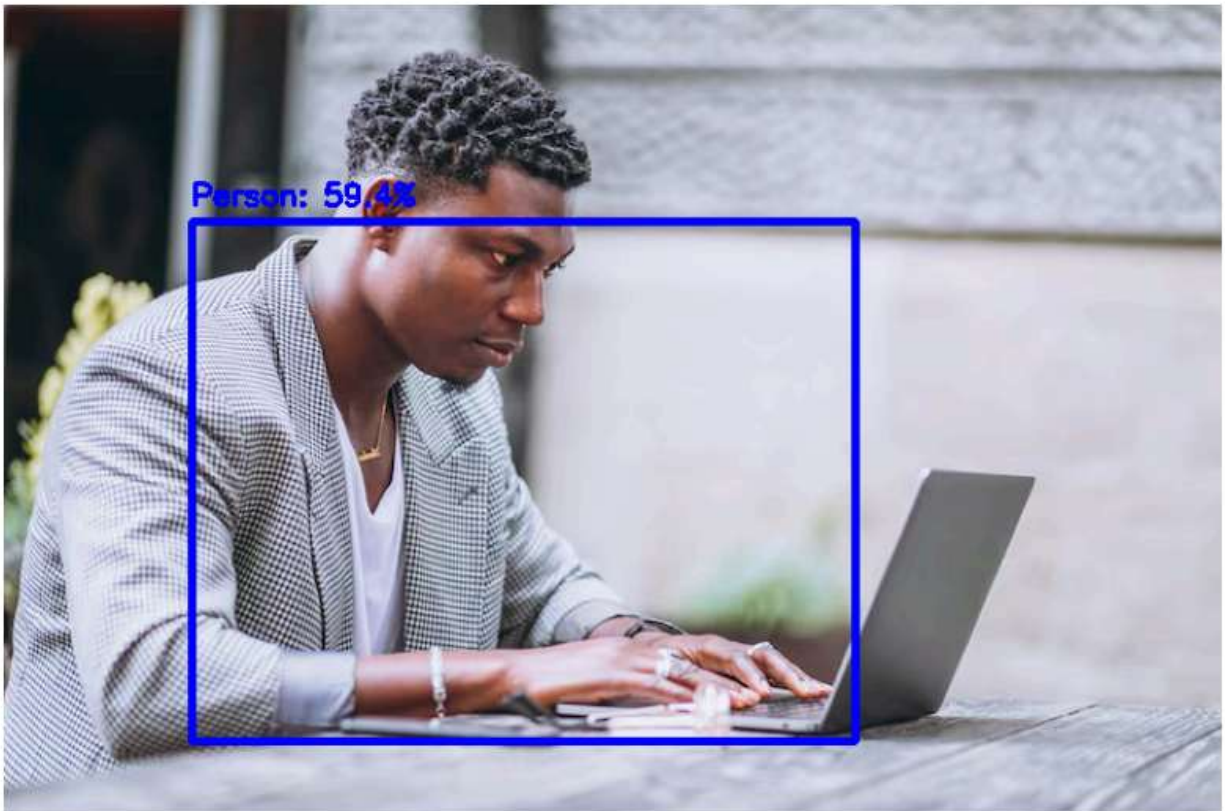
verify_image("test1.jpg", confidence_floor=0.30, nms_overlap_limit=0.3)

```

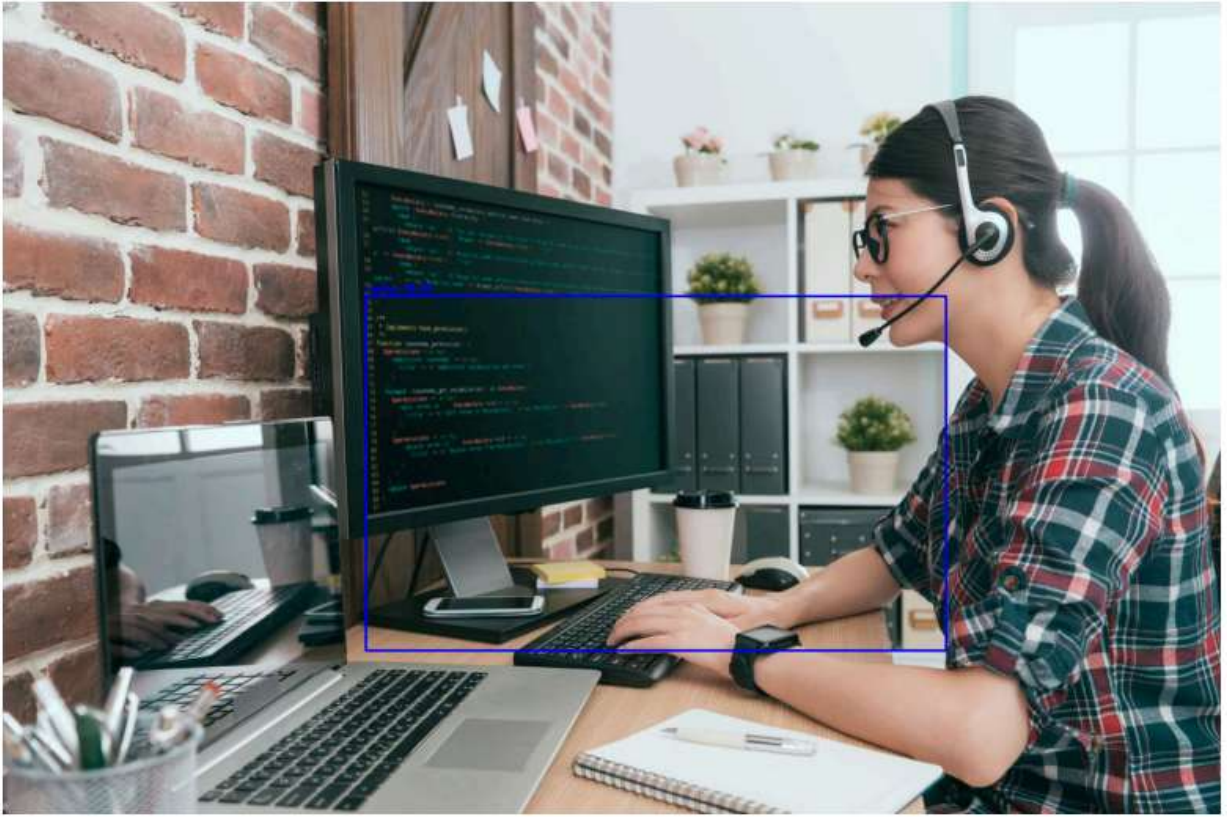
Done! Cleaned up and visualized 1 custom localized objects.

```
verify_image("test2.jpg", confidence_floor=0.30, nms_overlap_limit=0.3)
```



Done! Cleaned up and visualized 1 custom localized objects.

```
verify_image("test3.jpg", confidence_floor=0.30, nms_overlap_limit=0.3)
```



Done! Cleaned up and visualized 1 custom localized objects.